CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering · Mini-Project Report

Student	Mayank Tanwar	**Roll number**	IMT2023012
Faculty	Prof. B. Thangaraju	**Date**	30 October 2026
GitHub	github.com/Mayanktan/calculator-devops	**Docker Hub**	hub.docker.com/r/mayanktan/calculator-devops

1. Overview

The project is a production-style Calculator REST API in Spring Boot with a fully automated delivery pipeline. Every push to `main` on GitHub is built, tested, packaged into a Docker image, pushed to Docker Hub, deployed with Docker Compose behind an Nginx reverse proxy, health-checked, and the result is emailed. The API offers `add`, `subtract`, `multiply`, `divide` and `power` under `/api/v1` and returns `{"operation", "a", "b", "result"}`; divide by zero and missing or non-numeric parameters return HTTP 400 with a structured JSON error, and `/actuator/health` reports UP or DOWN.

Tools: Java 21, Spring Boot 3.3.4 (Web, Actuator, Validation), Maven 3.9, JUnit 5 and Spring MockMvc, Git and GitHub, Jenkins LTS 2.528.2 with a declarative `Jenkinsfile`, Docker Engine 29.5 (Colima), Docker Compose, Nginx 1.27, Docker Hub, and the Email Extension plugin over Gmail SMTP.

2. Architecture

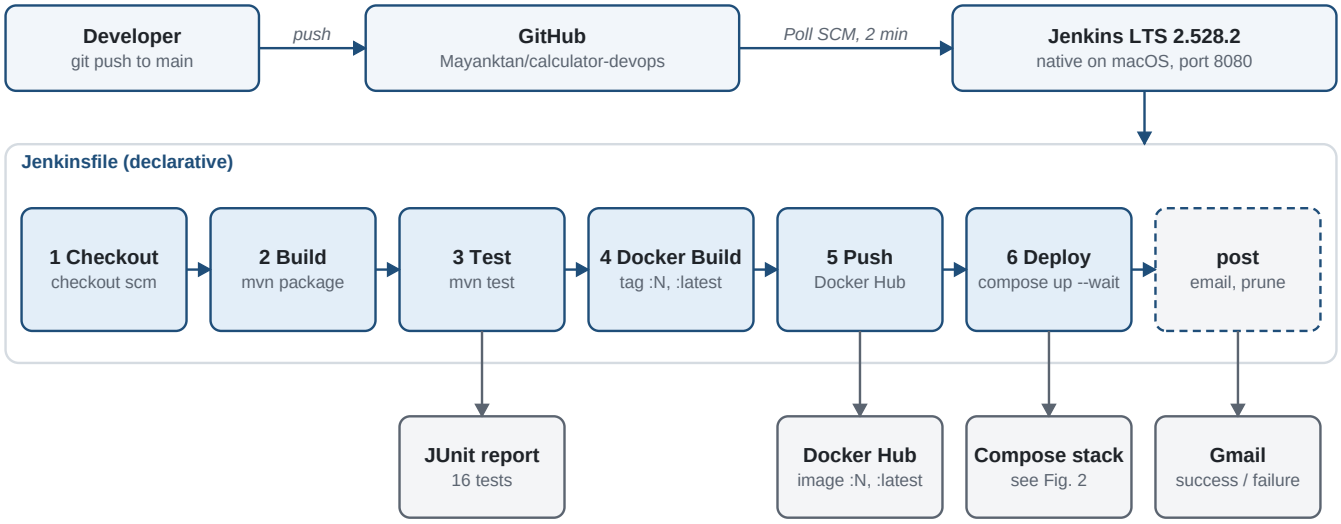


Fig. 1 Pipeline: a push to GitHub is picked up by Jenkins, which runs six stages; the side outputs are the test report, the pushed image, the deployed stack and the email.

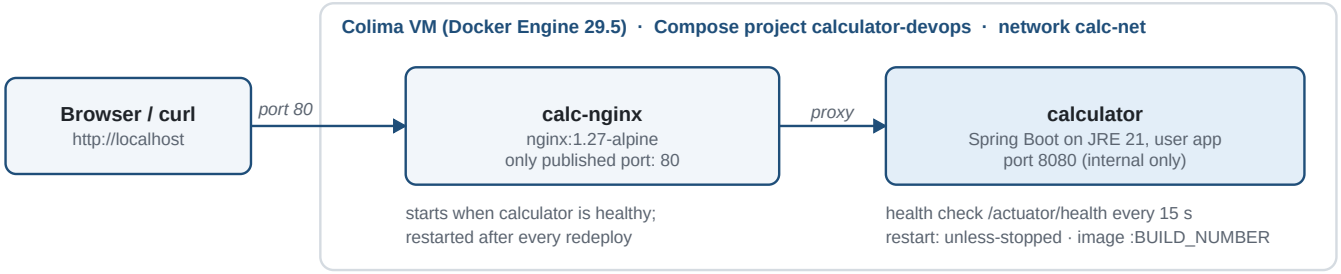


Fig. 2 Deployment: only Nginx is reachable from the host; the application's port 8080 stays inside the `calc-net` network.

3. Pipeline stages

Stage	What it does	Fails the build when
1. Checkout	`checkout scm` gets the latest commit of `main` (the implicit checkout is off, so there is exactly one)	GitHub cannot be reached
2. Build	`mvn -B -ntp clean package -DskipTests` compiles and builds the executable JAR	the code does not compile
3. Test	`mvn test`; the `junit` step publishes `target/surefire-reports/*.xml`, even when tests fail	any test fails
4. Docker Build	`docker build` tagged `mayanktan/calculator-devops:$BUILD_NUMBER` and `:latest`	the image cannot be built
5. Push	`withCredentials(dockerhub-creds)`: `docker login --password-stdin`, push both tags, logout	login or push is rejected
6. Deploy	`docker compose up -d --remove-orphans --wait`, then `/actuator/health` through Nginx (up to 12 tries, 5 s apart)	the app is not healthy within a minute
post	HTML email to the student on success and on failure; prunes old dangling images	never

- **Trigger:** Poll SCM every 2 minutes, which the problem statement accepts. Jenkins listens only on localhost without a login, so it is not exposed to the internet for a webhook.
- **Credentials:** the Docker Hub access token and the Gmail App Password are stored in the Jenkins credentials store; the Jenkinsfile only refers to their IDs, and the values are masked in the log.
- **Re-runnable and idempotent:** every build pushes a unique tag, `compose up -d` recreates only what changed, the Compose project name is fixed, and `disableConcurrentBuilds()` stops two deploys from overlapping.

4. Containerization and deployment

Requirement	Implementation
Multi-stage Dockerfile	`maven:3.9.9-eclipse-temurin-21` builds the JAR; `eclipse-temurin:21-jre-alpine` runs it and copies only the JAR
Best practices	`pom.xml` copied first so dependencies are cached; non-root user `app` (uid 100); `EXPOSE 8080`; `.dockerignore`; image 331 MB (about 91 MB compressed)
Single `docker run`	`docker run -p 8080:8080 mayanktan/calculator-devops` answers on the mapped port
Compose stack	network `calc-net`; health check on `/actuator/health`; `restart: unless-stopped`; environment variables `SPRING_PROFILES_ACTIVE` and `JAVA_TOOL_OPTIONS`
Second component	Nginx reverse proxy, the only published port (80); restarted after every redeploy
One command	`docker compose up -d` starts the whole stack

5. Testing and version control

- **Tests:** 8 JUnit 5 unit tests cover every operation, including divide by zero; 8 Spring MockMvc integration tests call the real endpoints and check status codes, JSON fields, the 400 error bodies and the health endpoint. `mvn test` writes the JUnit XML that Jenkins publishes.
- **Git and GitHub:** public repository with a README (build, test, run and API calls); `.gitignore` committed first (excludes `target/`, IDE files and secrets); conventional commit messages; five feature branches merged through five pull requests; no credentials in the repository.

6. Results

Build	Started by	Result	Time	Tests	What it showed
#1	first run	SUCCESS	180 s	16/16	all six stages; first upload of every image layer
#2	Build Now (web page)	SUCCESS	37 s	16/16	re-run over a live stack with cached layers
#3	SCM change: new page	SUCCESS	40 s	16/16	started 77 s after the push; new page live
#4	SCM change: broken test	FAILURE	17 s	15/16	stopped at Test; later stages skipped
#5	SCM change: revert	SUCCESS	40 s	16/16	back to green

Build #4 failed **on purpose**: one assertion was changed to expect 2 + 3 = 6, to prove that a failing test stops the pipeline. Docker Build, Push and Deploy were skipped, no image was pushed for build 4, the running application stayed on build 3, and the failure email arrived; build #5 reverted the change. Every build emailed its result. Later pushes, such as the commits that added this report (builds #6 and #7), also passed all six stages.

7. Challenges and fixes

Problem	Cause	Fix
Health check through Nginx returned **HTTP 400**, so every Deploy would have failed	The `/actuator/` route sent no Host header, so Nginx used the upstream name `calc_backend`; Tomcat rejects the underscore	Upstream renamed to `calc-backend`; proxy headers set once for every route
Only 8 of 16 tests ran	Surefire runs only `*Test.java` by default, so `CalculatorControllerIT` was skipped	Explicit Surefire includes for `*Test.java` and `*IT.java`
Nginx kept old config after a redeploy	Compose replaced only the app container; Nginx reads its config and resolves the app only at start	`depends_on: restart: true` restarts Nginx after each app update
`release version 21 not supported` in Jenkins	The Mac's default JDK is 17, and Jenkins started by launchd had no Homebrew tools on its PATH	Global Jenkins variables `JAVA_HOME` (JDK 21) and `PATH+EXTRA`
Docker broken after Docker Desktop was removed	`~/.docker` still used the Desktop credential helper and had dead plugin links	Backed up and cleaned `~/.docker`; installed Colima with Compose and Buildx
Pre-demo script always reported success	Under `set -e`, a failing `check && echo OK` does not stop bash	One command per line, so the first failed check stops the script

8. Submission checklist

Deliverable (problem statement, section 5)	Where (screenshot files are in `docs/screenshots/`)
GitHub repository with source, Jenkinsfile, Dockerfile, docker-compose.yml	https://github.com/Mayanktan/calculator-devops
Docker Hub image link (public)	https://hub.docker.com/r/mayanktan/calculator-devops
Report: architecture diagram, stage descriptions, challenges	This PDF (sections 2, 3 and 7); `docs/report.pdf` in the repository
Screenshot: green Jenkins pipeline run	Fig. 3; `required/1-jenkins-pipeline-green.png`
Screenshot: email notification	Figs. 4 and 5; `required/2-email-success-build5.png`, `required/2-email-failure-build4.png`
Screenshot: docker ps showing running containers	Fig. 6; `required/3-docker-ps.png`
Screenshots: sample API responses	Fig. 7; `required/4-api-responses.png`
Live demo: push a change, pipeline auto-deploys it	In class; `scripts/preflight.sh` checks the setup first

Future work (capstone): Ansible provisioning, Kubernetes using the Actuator probes already enabled, Prometheus and Grafana on the Actuator metrics, a secured Jenkins with a GitHub webhook, and a multi-architecture image.

9. Screenshots

9.1 Required by the problem statement

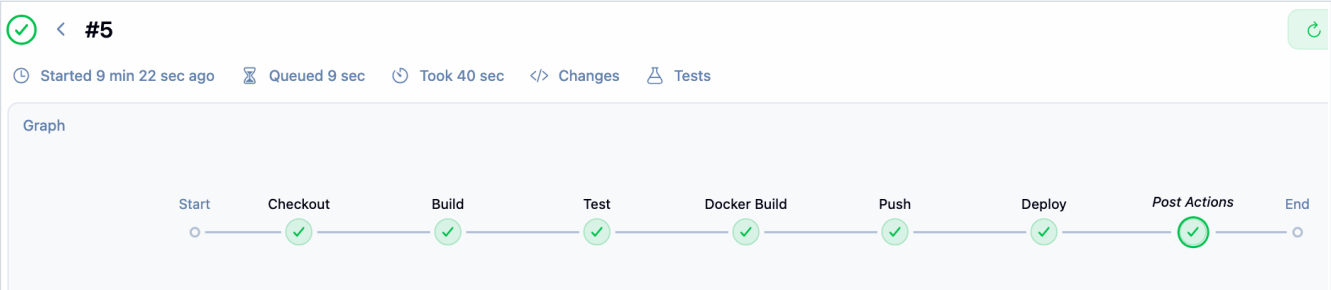


Fig. 3 Green Jenkins pipeline run: build #5, all six stages passed, then the post step sent the email.

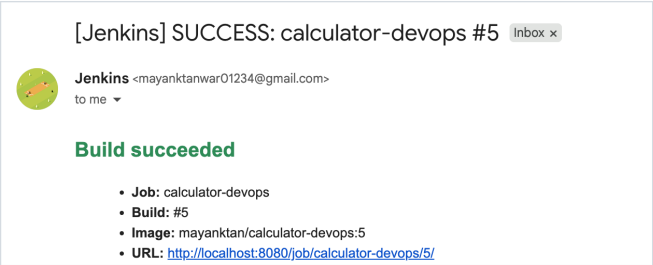


Fig. 4 Email notification: success (build #5).

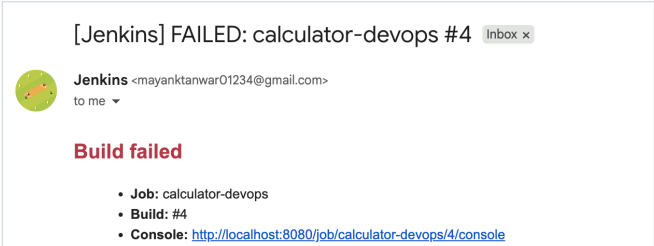


Fig. 5 Email notification: failure (build #4).

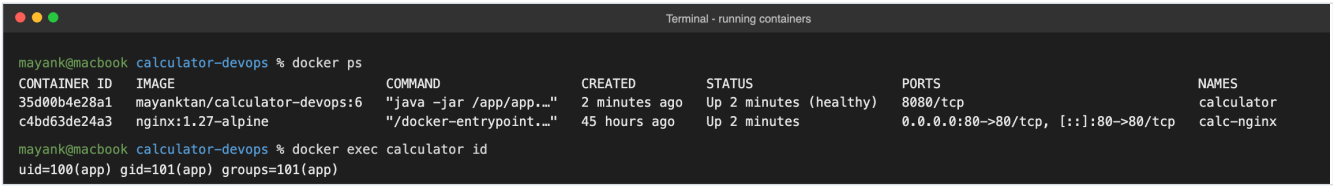


Fig. 6 docker ps showing the running containers: calculator healthy on internal port 8080, Nginx published on port 80; the app runs as non-root uid 100.

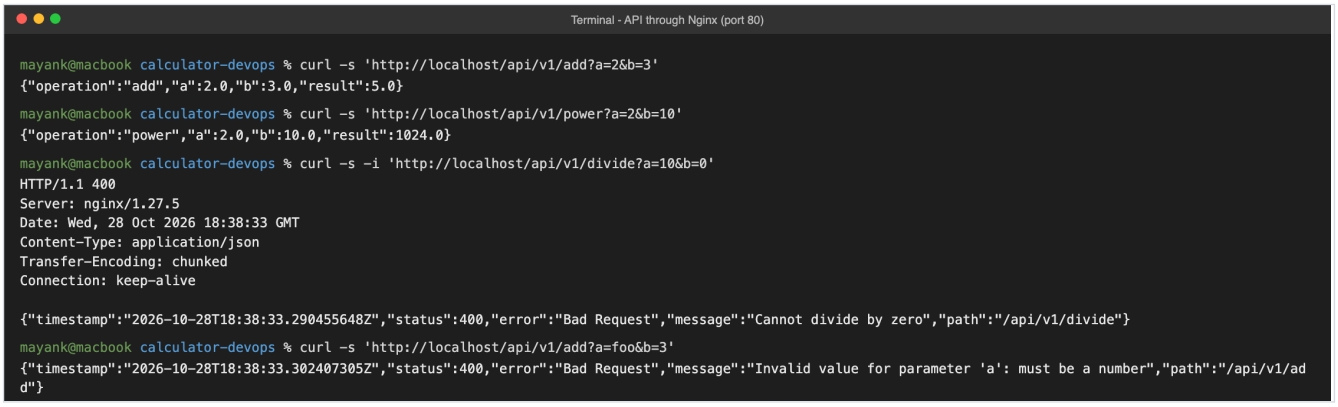


Fig. 7 Sample API responses through Nginx: results, and HTTP 400 JSON errors for divide by zero and a non-number.

9.2 Supporting evidence

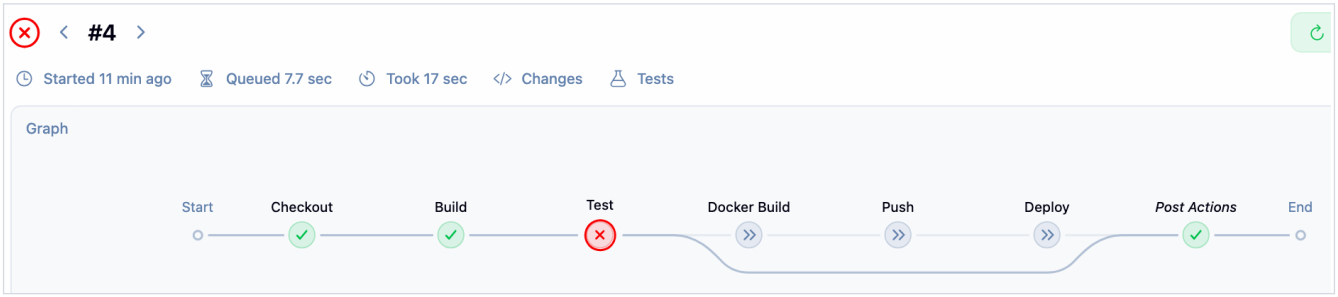


Fig. 8 Build #4: the Test stage failed on purpose, so Docker Build, Push and Deploy were skipped.

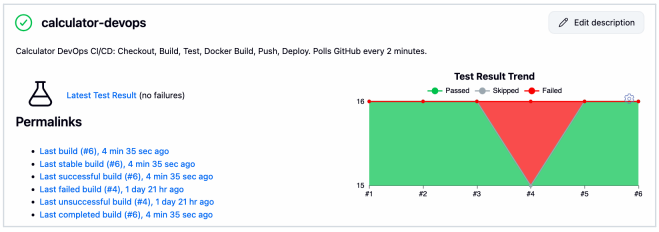


Fig. 9 Jenkins job: test result trend across builds.

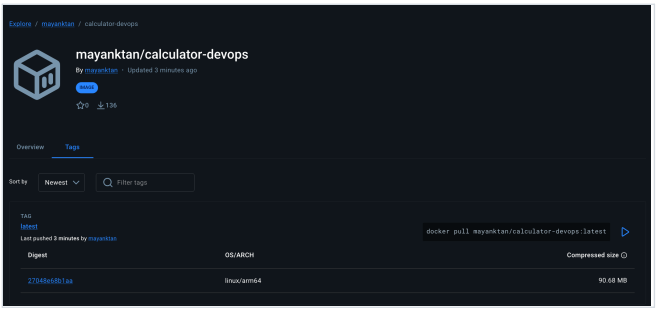


Fig. 10 Docker Hub: public image, one tag per build plus `latest`.

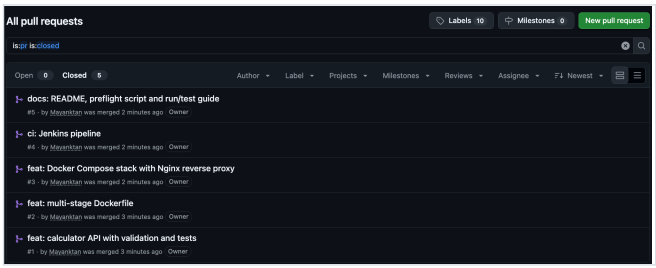


Fig. 11 GitHub: the five merged pull requests.

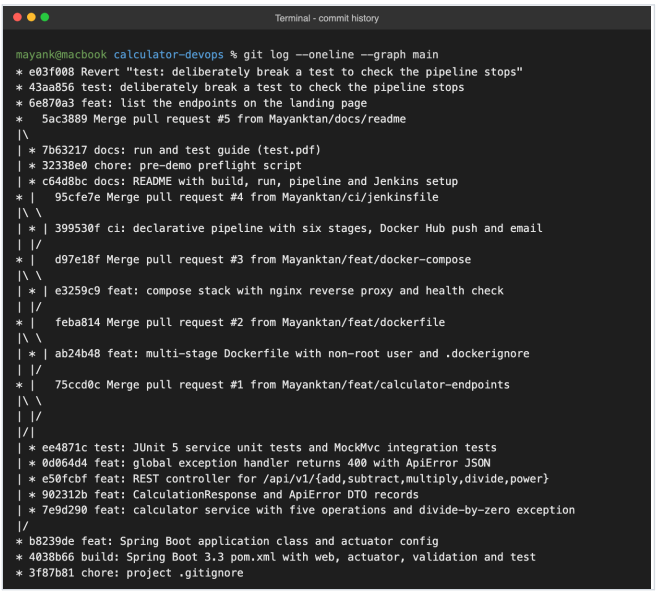


Fig. 12 Commit history of `main`: feature branches merged through pull requests, then the pipeline tests.